

BRIDS

INVESTOR / PARTNER BRIEF

Roadmap Tecnico e Investor Brief

BRIDS App Technical Roadmap and Investor Brief

BRIDS is a Solana-first real estate investment marketplace with active product surfaces across marketplace discovery, wallet/federated authentication, protected investor dashboards, admin operations, checkout, purchase minting, compliance, staking, notifications, and observability.

Snapshot date	2026-06-05
Audience	Investors, strategic partners, founders, product, engineering, and operations
Document type	Technical and product-readiness brief. Not legal, tax, regulatory, investment, or compliance advice.

Contents

1 Español	1
1.1 Proposito	1
1.2 Resumen Ejecutivo	1
1.3 Tesis de Producto	2
1.4 Arquitectura Actual	2
1.5 Que Esta Hecho	2
1.6 Matriz de Madurez	9
1.7 Brechas Tecnicas Clave	10
1.8 Roadmap Propuesto	11
1.9 Traduccion a Pitch Deck	12
1.10 Claims Recomendados para Inversionistas	13
1.11 Indice de Evidencia	14
1.12 Acciones Inmediatas	14
2 English	15
2.1 Purpose	15
2.2 Executive Summary	15
2.3 Product Thesis	15
2.4 Current Architecture	16
2.5 What Is Done	16
2.6 Maturity Matrix	23
2.7 Key Technical Gaps	24
2.8 Proposed Roadmap	24
2.9 Pitch Deck Translation	26
2.10 Recommended Investor Claims	27
2.11 Evidence Index	27
2.12 Immediate Next Actions	28

1 Español

1.1 Proposito

Este documento resume que tiene hoy la app de BRIDS, que esta parcialmente construido y que deberia construirse hacia adelante. Esta escrito para dos audiencias:

- Inversionistas y aliados estrategicos que necesitan entender madurez tecnica, readiness de producto y ruta de ejecucion.
- Equipos internos de producto, ingenieria y operaciones que necesitan un roadmap compartido para las siguientes fases.

Este documento no es una opinion legal, tributaria, regulatoria ni de inversion. Es un brief tecnico y de readiness de producto basado en el estado del repositorio y la documentacion existente.

1.2 Resumen Ejecutivo

BRIDS es actualmente una aplicacion fullstack de marketplace inmobiliario Solana-first construida con Next.js. La superficie activa incluye sitio publico, marketplace, paginas de detalle de propiedades, autenticacion con wallet y federada, dashboards protegidos para inversionistas, operaciones administrativas, gestion de colecciones/contenido, checkout, purchase minting, compliance, referidos, staking/unstaking, notificaciones web push, infraestructura SEO/AI-readable y automatizacion de gobernanza.

1.3 Tesis de Producto

BRIDS esta construyendo una plataforma de inversion inmobiliaria regulada y wallet-aware, donde el inventario tokenizado puede ser descubierto, comprado, administrado y operado despues mediante un lifecycle transparente on-chain/off-chain.

El codigo actual soporta cuatro narrativas principales para inversionistas:

1. Descubrir oportunidades inmobiliarias tokenizadas mediante marketplace y detalle de propiedad.
2. Autenticarse con SIWS wallet-first o con una ruta federada de menor friccion que luego puede vincular una wallet.
3. Ejecutar compras crypto ya configuradas para recepcion en USDC, y sumar compra con tarjeta credito/debito mediante Sphere ramp bajo un modelo controlado de orden.
4. Operar holdings desde dashboards protegidos, compliance gates, referidos, staking/unstaking, rentas/portfolio y notificaciones.

La direccion tecnica es convertir esta base en una plataforma production-ready endureciendo identidad, compliance, pagos, autoridad on-chain, observabilidad y performance del marketplace.

1.4 Arquitectura Actual

Capa	Implementacion actual
Frontend	Next.js App Router, React 19, Tailwind, Motion 12, Mapbox/React Map GL, cobertura responsive E2E.
Backend	Route handlers de Next.js, servicios server-only en lib/, repositorios PostgreSQL, rutas idempotentes de compra/checkout.
Blockchain	Politica devnet Solana, Metaplex Core, Core Candy Machine, Umi, helpers de compatibilidad Solana Kit, owner-freeze/stake flows.
Auth	Modelo hibrido WorkOS + SIWS, wallet linking, account repository, derivacion de rol admin, rutas protegidas.
Datos	Migraciones SQL para marketplace entries, checkout, compliance, referidos, notificaciones, staking y sesiones de upload/edit.
Admin Ops	Admin shell con assets, collections, compliance, dashboards, notifications, treasury, sales, monitoring y minting.
Calidad	npm run validate, Vitest, Playwright, Synpress, docs governance, checks de routes/content/schema/SEO/operability.

1.5 Que Esta Hecho

1.5.1 Sitio Publico y Marketplace

Estado: implementado, con follow-ups de performance y bugs de mapa.

Capacidades implementadas:

- Landing y paginas institucionales publicas.
- Marketplace con rutas de lista, mapa y detalle.
- Read model de marketplace respaldado por registros persistidos en DB con fallback de snapshot.
- Mapa de marketplace con Mapbox/React Map GL y fallback list-only si la configuracion del mapa no esta disponible.
- Detalle de propiedad con media, resumen de inversion, economia, fees/return, documentos/blockchain, gobernanza, informacion de propiedad y preview de Google Maps.
- SEO, structured data, feeds, endpoints AI-readable, llms.txt, ai.txt, rutas de knowledge y serializers de contenido.

Limitaciones actuales:

- Falta implementar la animacion inicial de entrada con Motion 12 para reforzar la primera impresion de la app, cuidando performance, accesibilidad y preferencias de movimiento reducido.
- La performance del mapa sigue siendo una frontera conocida, especialmente JavaScript/LCP/TBT en mobile.
- Los bugs conocidos del mapa deben corregirse antes de investor demo y private beta readiness.
- Siguen pendientes trabajos de mapa en cola P2: lazy Mapbox boundary, Web Vitals recheck, validacion de coordenadas y degraded-state polish.

Evidencia principal:

- app/marketplace/page.tsx
- app/marketplace/[id]/page.tsx
- components/marketplace/*
- lib/property-marketplace-server.ts
- docs/features/feature-app-create-a-marketplace-3d-visual-bri-164.md

1.5.2 Autenticacion y Modelo de Cuenta

Estado: foundation implementada, requiere hardening de storage productivo.

Capacidades implementadas:

- Autenticacion wallet-first con SIWS.
- Foundation de login federado con WorkOS.
- Flujo explicito de wallet linking.
- Account repository server-side.
- Modelo de request authority que diferencia acceso account-only de autoridad financiera/admin wallet-authenticated.
- Proteccion de rutas admin mediante roles derivados de wallet.
- Captura de referral intent durante auth.

Limitaciones actuales:

- El almacenamiento de sesion documentado es process-local/in-memory y debe moverse a un backend persistente compartido antes de escalar horizontalmente en produccion.
- Las politicas de consolidacion/recuperacion de cuentas siguen siendo flujos futuros controlados; no hay merges automaticos silenciosos.

Evidencia principal:

- app/api/auth/*
- lib/auth.ts
- lib/auth-store.ts
- lib/auth-session.ts
- lib/accounts/repository.ts
- docs/auth-flow.md
- docs/session-model.md
- docs/features/feature-shared-hybrid-auth-workos-wallet-bri-154.md

1.5.3 Checkout, Ordenes y Pagos

Estado: parcialmente implementado.

Capacidades implementadas:

- APIs de carrito y orden.
- Modelo de datos de orden y payment attempt.
- Purchase flow crypto ya implementado y configurado para que BRIDS reciba USDC.
- Integracion con reserva/consumo de onboarding reward.

Limitaciones actuales:

- Falta implementar de punta a punta la funcion de compra con tarjeta credito/debito mediante Sphere ramp.
- Despues de implementar Sphere ramp, se debe integrar la experiencia para que compra con tarjeta y compra crypto se comporten como un flujo coherente.
- El contrato final de Sphere ramp, webhook model, settlement mapping y soporte operativo para compras con tarjeta siguen pendientes.

Evidencia principal:

- app/checkout/page.tsx
- app/api/checkout/*
- lib/checkout-service.ts
- lib/checkout-repository.ts
- db/migrations/018_checkout_dual_payment.sql

1.5.4 Purchase Minting

Estado: purchase crypto implementado y configurado para recepcion USDC, con hardening productivo pendiente.

Capacidades implementadas:

- APIs de quote, challenge, prepare y submit.
- Emision y consumo de anti-bot challenge.
- Rate limiting e idempotency storage.
- Validacion de Candy Machine guard.
- Soporte de third-party signer.
- Modelo de moneda de pago SOL/USDC.
- Sistema configurado para que BRIDS reciba USDC.
- Confirmacion de transaccion enviada.
- Verificacion de asset post-submit y soporte de reconciliacion.
- Guard de compliance restriction en rutas de purchase challenge/prepare/submit.

Limitaciones actuales:

- Jupiter queda como extension propuesta para ampliar crypto input, pero el flujo base crypto y la recepcion USDC ya existen.
- El lanzamiento productivo requiere politica final de treasury/payment destination, plan de migracion devnet-to-production y evidencia live de pagos/assets.
- La gobernanza del repo espera aceptacion blockchain/NFT en devnet salvo que una politica productiva sea aprobada explicitamente.

Evidencia principal:

- app/api/purchase/*
- lib/purchase-service.ts
- lib/purchase-anti-bot.ts
- lib/purchase-attempts-repository.ts
- lib/purchase-webhook-reconciliation.ts
- docs/purchase-tracing.md
- docs/devnet-proof.md

1.5.5 Admin Assets y Collections

Estado: foundation implementada con mejoras continuas de workflow.

Capacidades implementadas:

- Formulario admin de creacion de asset.
- Upload workflow con signed URLs, client upload, finalize, edit sessions, orphan reconciliation, CDN purge y soporte de PDF brief.
- Import jobs e import preview.

- Lista/detalle de admin collections, superficies de edicion, documents, gallery, property information, location maps y health workspace.
- Creacion de marketplace entry desde flujo admin.
- Collection health read models y soporte para manual review.

Limitaciones actuales:

- Algunas pantallas admin requieren mejoras continuas de usabilidad, consistencia operativa y preparacion para uso diario por el equipo interno.
- Antes de un lanzamiento externo se deben escribir playbooks no tecnicos y procedimientos productivos de operacion.

Evidencia principal:

- app/admin/assets/new/page.tsx
- app/admin/collections/*
- app/api/admin/assets/*
- app/api/admin/collections/*
- components/admin/*
- lib/admin/*
- db/migrations/002_asset_upload_contracts.sql
- db/migrations/003_asset_import_jobs.sql
- db/migrations/020_asset_upload_edit_sessions.sql

1.5.6 Admin Dashboard y Operaciones

Estado: superficie admin implementada, requiere modulos operativos criticos.

Capacidades implementadas:

- Admin shell con navegacion hacia dashboard, assets, collections, compliance, notifications, treasury, sales, monitoring y minting.
- Paneles base para resumen ejecutivo, ventas, treasury, distributions, notifications y monitoring.
- Rutas admin protegidas por rol server-side.

Limitaciones actuales:

- Falta implementar el modulo de tesoreria usando Squads para que el equipo pueda ver balances, movimientos, aprobaciones y evidencia operacional desde el admin.
- Falta implementar el modulo de distribucion que integra Squads con el modulo freeze/unfreeze, para conectar elegibilidad, tesoreria, distribuciones y estado de claims.
- Falta mejorar el modulo de Resumen del dashboard admin para mostrar KPIs accionables: funnel de inversionistas, conversion de KYC, ventas, fondos recibidos, estado de tesoreria, distribuciones pendientes, claims, compliance queue, salud de colecciones y alertas operativas.
- El modulo de notificaciones debe integrarse con CRM para campanas, seguimiento de posibles clientes, segmentacion, estado de contacto y trazabilidad de outreach.
- El modulo de cumplimiento debe integrar KYC, Stripe Identity, persistencia de datos de usuarios en la base propia y permisos RBAC de compliance.

Evidencia principal:

- app/admin/dashboard/page.tsx
- app/admin/treasury/page.tsx
- app/admin/distributions/page.tsx
- app/admin/notifications/page.tsx
- app/admin/monitoring/page.tsx
- components/admin/executive-dashboard.tsx
- components/admin/treasury-console.tsx
- components/admin/distributions-console.tsx
- components/admin/admin-notification-campaign-console.tsx

1.5.7 Metaplex Core, Candy Machine y NFT Operations

Estado: implementado en devnet con backlog de authority lifecycle hardening.

Capacidades implementadas:

- Minting de colecciones y assets Metaplex Core.
- Flujos Core Candy Machine deploy/submit/status/metadata/mint prepare/snapshot finalize.
- Payload Economic AppData para campos de revenue share/yield.
- Backend de authority lifecycle para transfer_delegate y appdata_authority.
- Authority registry y audit events.
- Evidencia devnet para operaciones de collection, asset, AppData y authority.

Limitaciones actuales:

- Authority lifecycle tiene endpoints backend, pero falta UI administrativa operable.
- Endpoints de lectura de registry/audit estan propuestos pero no implementados.
- La validacion de evidencia Squads hoy valida estructura/evidencia declarativa, no prueba completa on-chain de proposal execution.
- Falta backfill de colecciones legacy al authority registry.

Evidencia principal:

- app/api/admin/core-candy-machine/*
- app/api/admin/metaplex-core/*
- lib/core-candy-machine-admin.ts
- lib/core-authority-lifecycle.ts
- lib/metaplex-core-admin.ts
- docs/nft-spec.md
- docs/authority-model.md
- docs/state-machine.md
- docs/devnet-proof.md
- docs/rotation-spec.md

1.5.8 Compliance, KYC, AML y Suspension

Estado: modelo operativo implementado, requiere hardening de provider/live policy.

Capacidades implementadas:

- Proyeccion de compliance status: pending KYC, pending AML, pending review, fully verified, restricted AML, suspended.
- Queue admin de compliance cases.
- KYC decision, AML decision, suspend/unsuspend y notes.
- Modelo de audit event para mutaciones admin de compliance.
- Enriquecimiento AML mediante servicio orientado a Helius.
- Guardrails financieros que bloquean usuarios restricted/suspended en rutas de compra.

Limitaciones actuales:

- La integracion live de KYC/AML provider y la politica operacional de compliance deben finalizarse antes de produccion.
- Falta un modulo que recupere datos desde Stripe Identity y los guarde de manera persistente en la base de datos propia de BRIDS, bajo una politica aprobada de minimizacion, seguridad y retencion de datos.
- Falta implementar una vista admin para que el equipo autorizado pueda consultar los datos KYC de clientes y su estado de revision.
- El RBAC debe extenderse con un rol especializado de compliance reviewer/compliance agent, separado del rol admin general, para operar revisiones KYC/AML con permisos acotados.
- SLAs de manual review, ownership de escalamiento y data retention deben documentarse para operaciones e inversionistas.

Evidencia principal:

- app/admin/compliance/page.tsx
- app/api/admin/compliance/*
- app/api/internal/compliance/aml/screen/route.ts
- lib/compliance/*
- db/migrations/012_profile_kyc_compliance.sql
- db/migrations/013_aml_screening_enrichment.sql
- db/migrations/014_compliance_notes.sql
- docs/architecture.md

1.5.9 Dashboard Protegido para Inversionistas

Estado: superficie implementada, algunos modulos requieren integracion con datos financieros reales.

Capacidades implementadas:

- Protected layout y dashboard.
- Modulos de portfolio, rentas, referrals, stake, profile, history y account support.
- Profile capture, KYC module, onboarding reward reminders y guided tour.
- Referral landing y dashboard services.
- Ruta de web push enrollment ligada a wallet.

Limitaciones actuales:

- El resumen del user dashboard debe alimentarse con datos reales de los modulos de portfolio, compras, rentas/claims, compliance, staking y notificaciones.
- Mi Portafolio debe alimentarse con datos reales desde la base de datos y reconciliacion on-chain, no con snapshots o placeholders.
- Rentas / Claim debe implementarse para que el usuario pueda cobrar desde alli, solicitar transferencias, ver estados de claim/distribucion y dar seguimiento con datos persistentes desde la base de datos propia.
- Historial debe mostrar operaciones reales del usuario: compras, staking/unstaking, claims, transferencias, compliance events relevantes y transacciones de wallet reconciliadas, alimentandose de los demas modulos y de datos persistentes en la base de datos.
- Statements, documentos tributarios, historial de distribucion y reportes descargables siguen como trabajo futuro salvo que vivan fuera de este repo.

Evidencia principal:

- app/protected/*
- components/dashboard/*
- lib/referrals/*
- lib/onboarding-reward-service.ts
- db/migrations/023_referral_wallet_first_schema.sql
- db/migrations/024_onboarding_profile_completion_rewards.sql

1.5.10 Staking y Unstaking

Estado: flujo devnet-oriented owner-freeze implementado.

Capacidades implementadas:

- Pagina protegida de stake.
- Descubrimiento de inventario via DAS para assets propios.
- Flujo stake/unstake prepare y submit.
- Construccion de transacciones freeze/thaw.
- Persistencia de attempts y reconciliacion.
- Cobertura E2E responsive para superficie protected stake.

Limitaciones actuales:

- Falta implementar el servicio de reconciliacion de eventos stake/unstake y preparacion de distribuciones: validacion por RPC canonico, Helius como observador/indexador, proyeccion interna de eventos validados, lectura de KYC/perfil para elegibilidad y lectura de tesoreria/Squads o snapshot financiero del periodo.

- Falta implementar el pipeline por periodo para calcular tiempo elegible, versionar politica de distribucion, manejar estados `sync_pending/reconcile_pending`, generar archivo de salida para claim o ejecucion posterior, e imponer idempotencia/reintentos/reconciliacion obligatoria.
- Falta implementar el microservicio de distribucion y claim lifecycle: estados `pending, claimed, failed, disputed`, API para UI/backoffice, reconciliacion de claims con evidencia de transaccion y reprocesamiento/recalculo por periodo.
- Falta implementar la arquitectura de trazabilidad/auditoria: snapshots de tesoreria, estados NFT, corridas de distribucion, items de distribucion y audit logs consultables para UI y auditoria externa.
- La semantica productiva debe definirse claramente: que significa staking financieramente, cuando acumula rewards, como se reclama, y como interactua con compliance y transferencia de propiedad.
- Copy final para inversionistas, estados de claim/distribucion y encuadre legal/regulatorio deben cerrarse antes del launch.

Evidencia principal:

- `app/protected/stake/page.tsx`
- `app/api/protected/profile/stake/*`
- `components/dashboard/stake-module.tsx`
- `lib/stake-service.ts`
- `db/migrations/031_stake_profile_persistence.sql`
- `docs/STAKE_AUDIT.md`

1.5.11 Notificaciones y PWA

Estado: foundation implementada con rollout gates.

Capacidades implementadas:

- Web push subscriptions.
- Transactional delivery jobs.
- APIs internas de enqueue/process.
- Preview/send de admin notification campaigns.
- Notification health checks.
- PWA capability UI e installability shell.
- Contrato de suscripcion wallet-bound.

Limitaciones actuales:

- La entrega productiva de push requiere configuracion final VAPID/env, audience policy, rate limits y monitoreo operativo.
- Las admin campaigns deben seguir protegidas por preview hash y blocked-reason checks.

Evidencia principal:

- `app/api/notifications/*`
- `app/api/internal/notifications/*`
- `app/api/admin/notifications/*`
- `components/admin/admin-notification-campaign-console.tsx`
- `lib/notifications/*`
- `db/migrations/027_web_push_subscriptions.sql`
- `db/migrations/028_web_push_delivery_jobs.sql`
- `db/migrations/029_admin_push_campaigns.sql`

1.5.12 Observabilidad, Gobernanza y Calidad

Estado: controles internos fuertes implementados.

Capacidades implementadas:

- Health endpoint y rutas admin de monitoring.
- Operability logging store y analytics event route.
- Tests de security headers.

- Validacion de docs governance.
- PR policy y required docs checks.
- Scripts RFC y Linear planning.
- Cobertura Playwright/Synpress para browser y wallet-critical flows.
- Gate superior npm run validate.

Limitaciones actuales:

- Production readiness para inversionistas debe incluir uptime targets, alerting ownership, incident response y evidencia de backup/restore.
- La validacion local debe complementarse con smoke checks especificos de deployment para QA/RC/prod.

Evidencia principal:

- app/api/health/route.ts
- app/api/admin/monitoring/*
- lib/observability/*
- scripts/ci/*
- scripts/rfc-new.js
- scripts/linear-plan.js
- docs/governance/*
- tests/*
- e2e/*

1.6 Matriz de Madurez

Dominio	Madurez	Lectura para inversionistas	Siguiente paso principal
Marketplace discovery	Construido	Existe una superficie real de producto.	Animacion inicial con Motion, bugs de mapa, performance mobile y lazy Mapbox boundary.
Detalle de propiedad	Construido	Los assets pueden presentarse con inversion, documentos y gobernanza.	Completar datos finales y copy de compliance.
Wallet auth	Construido	Existe un modelo fuerte de autoridad wallet.	Persistent production session store.
Federated auth	Foundation construida	Existe ruta de onboarding de menor friccion.	Completar operaciones WorkOS productivas y recovery flows.
Checkout	Parcial	Existe modelo de orden; falta Sphere ramp para tarjeta.	Implementar Sphere ramp completo y unificar tarjeta + crypto.
Crypto purchase mint	Construido	Flujo crypto implementado y configurado para recibir USDC.	Hardening productivo, treasury policy, evidencia final y Jupiter opcional.
Admin asset ops	Construido	El equipo interno puede administrar inventario y metadata.	Playbooks operativos y hardening productivo.
Admin dashboard	Parcial	Existe shell admin; faltan modulos operativos clave.	Tesoreria Squads, distribuciones freeze/unfreeze, notificaciones CRM y KPIs accionables.
NFT/admin minting	Construido en devnet	Existe lifecycle Core asset.	Authority UI y endpoints de lectura.

Dominio	Madurez	Lectura para inversionistas	Siguiente paso principal
Compliance	Foundation construida	Los compliance gates estan codificados en rutas de transaccion.	Persistencia Stripe Identity, vista admin KYC y rol RBAC de compliance.
Investor dashboard	Superficie construida	Existe UX de cuenta y holdings.	Resumen, portafolio, rentas/claim e historial con datos reales.
Staking	Base construida en devnet	Existe asset action path; faltan distribuciones y auditoria.	Reconciliacion on-chain, claim state, distribuciones y trazabilidad.
Notifications	Foundation construida	Existe infraestructura de re-engagement.	Integracion CRM para campanas y seguimiento de leads.
Observability/QA	Foundation fuerte	Hay disciplina de ingenieria visible.	SLOs productivos, alerting y deployment smoke gates.

1.7 Brechas Tecnicas Clave

1. Persistent session storage

Mover auth/session state desde memoria local del proceso hacia un store productivo compartido. Esto es requerido para deployments multi-instancia, restarts y comportamiento de auth predecible.

2. Checkout y payment hardening

Implementar de punta a punta las compras con tarjeta credito/debito mediante Sphere ramp, definir contrato final de webhook/settlement y unificar el flujo de tarjeta con el purchase crypto ya implementado.

3. Treasury y politica de pago productiva

Validar USDC settlement productivo, definir payment destinations, treasury ownership, configuracion por ambiente y evidencia de auditoria antes de mover valor live. Jupiter queda como extension para broader crypto input.

4. Compliance operations

Implementar persistencia propia de datos recuperados desde Stripe Identity, vista admin de datos KYC por cliente, rol RBAC especializado para compliance reviewer/compliance agent, manual review workflow, data retention, support escalation y comportamiento para usuarios restringidos.

5. Authority lifecycle operations

Construir UI admin para rotate/revoke/emergency rotate, implementar endpoints de registry/audit, backfill de legacy collections y fortalecer validacion Squads/on-chain evidence.

6. Dashboard admin, tesoreria, distribuciones y CRM

Implementar tesoreria con Squads, distribuciones integradas con freeze/unfreeze, notificaciones conectadas a CRM para campanas/seguimiento de leads, y un resumen admin con KPIs accionables para operacion diaria.

7. Animacion inicial, performance y bugs del mapa

Implementar la animacion inicial con Motion 12, corregir bugs conocidos del mapa y diferir/lazy-load Mapbox en mobile preservando list-first usability, accesibilidad y calidad visual investor-grade.

8. User dashboard con datos reales y reporting para inversionistas

Conectar resumen, mi portafolio, rentas/claim, historial, downloadable statements y seguimiento de transferencias al ledger productivo final, reconciliacion de wallet/transacciones y fuentes persistentes de la base de datos.

9. Staking distributions and audit trail

Implementar el stack pendiente de staking/distribuciones: reconciliación canónica de eventos stake/unstake, pipeline por periodo, elegibilidad KYC, lectura de tesorería/Squads, distribution runs, claim lifecycle, APIs UI/backoffice y audit logs.

10. Production observability

Definir SLOs, deployment smoke checks, alerting ownership, incident response, backup/restore drills y release promotion evidence.

1.8 Roadmap Propuesto

1.8.1 Fase 0: Investor Demo Readiness

Target: 2 a 4 semanas.

Objetivo: hacer que la app actual sea demostrable con alta confianza y una historia clara de que existe hoy.

Entregables:

- Estabilizar demo environment desde develop o QA deployment.
- Preparar inventario semilla de marketplace con datos completos de detalle de propiedad.
- Implementar la animación inicial con Motion 12 para la entrada de la app.
- Corregir bugs conocidos del mapa que afecten la demo.
- Verificar happy paths de wallet login, marketplace, checkout preview, purchase quote, admin collections, compliance queue y protected dashboard.
- Producir demo script e investor FAQ desde este documento.
- Ejecutar `npm run validate` y evidencia Playwright crítica para la rama demo.
- Capturar screenshots/video para pitch deck.

Exit criteria:

- La demo puede ejecutarse end-to-end sin intervención local de ingeniería.
- Los gaps conocidos están etiquetados claramente como roadmap.

1.8.2 Fase 1: Production Foundation

Target: 4 a 8 semanas.

Objetivo: cerrar los gaps que bloquean una private beta controlada.

Entregables:

- Persistent session backend para SIWS/WorkOS/session composition.
- Implementar Sphere ramp completo para compras crédito/débito y confirmar webhook/idempotency behavior en deployment.
- Integrar el purchase crypto existente y la compra con tarjeta en una experiencia coherente de checkout/purchase.
- Mantener Jupiter como extensión propuesta para convertir crypto assets adicionales hacia USDC settlement.
- Compliance provider/live configuration, módulo de persistencia Stripe Identity, vista admin KYC y rol RBAC de compliance reviewer/compliance agent.
- Admin dashboard operativo: tesorería Squads, KPIs accionables, notificaciones CRM y preparación de distribuciones integradas con freeze/unfreeze.
- User dashboard con resumen, portafolio, rentas/claim e historial alimentados por datos reales persistidos.
- Production observability baseline: health checks, deployment smoke checks, alerting, incident owner, backup/restore plan.
- Animación inicial con Motion 12, marketplace mobile performance pass y Mapbox lazy boundary.

Exit criteria:

- Private beta puede soportar usuarios reales en ambiente controlado con auth y payment configuration productivas.

1.8.3 Fase 2: Regulated Transaction Readiness

Target: 8 a 12 semanas.

Objetivo: preparar la app para operaciones financieras/on-chain live sujeto a aprobacion legal y compliance.

Entregables:

- Politica final de treasury y payment destinations.
- Integracion coherente entre compra crypto existente y compra con tarjeta via Sphere ramp.
- Integracion Jupiter opcional para broad crypto input sobre la base USDC existente.
- Plan de aceptacion on-chain live o pre-live con signatures especificas por ambiente y fetched account state.
- UI de admin authority lifecycle y endpoints de lectura.
- Backfill de legacy authority registry.
- Compliance financial gates en todas las rutas que mueven valor.
- Ledger/distribution source of truth para portfolio, rentas, history y statements.
- Reconciliacion canonica de stake/unstake, preparation service de distribuciones, microservicio de distribucion, claim lifecycle y trazabilidad/auditoria para staking.
- Tesoreria Squads, modulo de distribuciones integrado con freeze/unfreeze, solicitudes de transferencia, seguimiento de claims y datos persistentes para admin/user dashboards.
- Security review de wallet linking, admin routes, payment webhooks, signer flows y authority rotation.

Exit criteria:

- La plataforma tiene controles auditables para identidad, compliance, treasury, admin authority y transaction execution.

1.8.4 Fase 3: Growth and Scale

Target: 3 a 6 meses despues de private beta foundation.

Objetivo: expandir inventario, mejorar conversion y soportar operaciones repetibles de inversionistas.

Entregables:

- Workflows multi-property con batch operations y review queues mas fuertes.
- Notificaciones integradas con CRM para campanas, seguimiento de posibles clientes, transaction state, compliance status, distributions y nuevas ofertas.
- Optimizacion de referidos y onboarding rewards.
- Admin analytics para funnel, sales, compliance y collection health.
- KPIs accionables de tesoreria, claims, CRM, conversion de KYC, ventas y health operacional.
- Expansion SEO/content del marketplace publico.
- Paquete de reporting institucional y data room exports.

Exit criteria:

- BRIDS puede presentar vision de producto, metricas operativas repetibles y growth loops controlados.

1.9 Traduccion a Pitch Deck

Este documento puede convertirse en un pitch deck con esta estructura:

1. Vision

- Marketplace de inversion inmobiliaria tokenizada con ownership wallet-aware y transparencia operacional.

2. Producto Hoy

- Marketplace, detalle de propiedad, dashboard protegido, admin operations, checkout, compliance, staking, notifications.

3. Moat Tecnico

- Hybrid auth, Solana/Metaplex Core integration, admin minting, compliance gates, governance automation, test coverage.

4. Arquitectura de Plataforma

- Next.js app, server domain layer, PostgreSQL, Solana devnet/on-chain services, WorkOS/SIWS, Sphere ramp, Jupiter, observability.
- 5. Readiness Actual
 - Marketplace, admin operations, auth, checkout, compliance, NFT, staking, notifications y observability ya estan representados en la app.
- 6. Risk Controls
 - Compliance statuses, purchase restrictions, idempotency, admin RBAC, authority registry/audit, docs governance.
- 7. Roadmap
 - Demo readiness, production foundation, regulated transaction readiness, growth and scale.
- 8. Uso de Fondos
 - Production hardening, compliance integration, payment/on-chain readiness, security review, marketplace performance, operations tooling.
- 9. Near-Term Milestones
 - Private beta, controlled transaction readiness, admin authority operations, investor ledger/distribution reporting.
- 10. Ask
 - Financiar el paso de working platform a marketplace de inversion compliant y production-grade.

1.10 Claims Recomendados para Inversionistas

Usar estos claims porque estan soportados por el repositorio:

- BRIDS tiene una app fullstack funcional con marketplace, admin, auth, checkout, compliance, NFT, staking, notifications y observability.
- El roadmap de pagos esta enfocado en implementar Sphere ramp completo para tarjeta y unificarlo con el purchase crypto ya configurado para USDC.
- La plataforma tiene implementacion Solana/Metaplex Core con evidencia devnet.
- El roadmap de compliance incluye persistencia propia de Stripe Identity, vista admin KYC y rol RBAC especializado para revisores.
- El roadmap operativo incluye tesoreria Squads, distribuciones integradas con freeze/unfreeze, CRM para notificaciones/campanas y dashboards alimentados por datos reales.
- El proceso de ingenieria incluye automated validation, docs governance, unit tests, route tests, Playwright y Synpress.
- El trabajo pendiente se enfoca en production hardening, compliance/payment readiness, controles operativos y escala.

Evitar estos claims hasta completar los items correspondientes:

- No afirmar mainnet production readiness.
- No afirmar operaciones reguladas live completamente habilitadas.
- No afirmar paridad completa tarjeta/crypto checkout hasta implementar Sphere ramp y la integracion unificada.
- No afirmar sesiones production-grade hasta implementar shared session backend.
- No afirmar operaciones completas de compliance hasta implementar persistencia Stripe Identity, vista admin KYC y rol RBAC especializado.
- No afirmar lifecycle completo de staking/distribuciones hasta implementar reconciliacion canonica, pipeline por periodo, servicio de distribucion, claim lifecycle y trazabilidad/auditoria.
- No afirmar dashboards operativos completos hasta conectar admin/user dashboards a tesoreria Squads, distribuciones, claims, CRM, reconciliacion y datos persistentes reales.
- No afirmar treasury, distributions, tax o investor statement infrastructure finalizados hasta completar ledger/reporting source of truth.

1.11 Índice de Evidencia

Repositorio y gobernanza:

- README.md
- package.json
- docs/governance/documentation-policy.md
- docs/governance/git-monorepo-policy.md
- docs/governance/security-quality-policy.md
- scripts/ci/check-required-docs.sh

Arquitectura y seguridad:

- docs/architecture.md
- docs/auth-flow.md
- docs/session-model.md
- docs/authority-model.md
- docs/state-machine.md
- docs/threat-model.md
- docs/rbac.md

Blockchain/NFT:

- docs/nft-spec.md
- docs/devnet-proof.md
- docs/rotation-spec.md
- lib/core-candy-machine-admin.ts
- lib/core-authority-lifecycle.ts
- lib/purchase-service.ts
- lib/stake-service.ts

Marketplace y admin:

- app/marketplace/*
- components/marketplace/*
- lib/property-marketplace-server.ts
- app/admin/*
- components/admin/*
- lib/admin/*

Checkout, compliance, notifications:

- app/api/checkout/*
- lib/checkout-service.ts
- app/api/admin/compliance/*
- lib/compliance/*
- app/api/admin/notifications/*
- lib/notifications/*

Testing:

- tests/*
- e2e/*
- npm run validate

1.12 Acciones Inmediatas

1. Revisar este documento con founders/product y suavizar cualquier claim de negocio que lo requiera.
2. Convertir la seccion de pitch deck en outline de deck con screenshots de la app.
3. Crear tickets de Fase 0 para demo readiness, bugs de mapa y evidence capture.

4. Crear tickets de Fase 1 para persistent sessions, implementacion completa de Sphere ramp, integracion tarjeta + crypto, persistencia Stripe Identity, vista admin KYC, rol RBAC de compliance y marketplace performance.
 5. Crear tickets de admin dashboard para tesoreria Squads, distribuciones freeze/unfreeze, KPIs accionables, notificaciones CRM y modulo de cumplimiento KYC.
 6. Crear tickets de user dashboard para resumen real, mi portafolio real, rentas/claim, solicitud de transferencias e historial reconciliado con wallet/modulos/base de datos.
 7. Crear tickets de staking/distribuciones para reconciliacion canonica de eventos, pipeline por periodo, elegibilidad KYC, tesoreria/Squads, servicio de distribucion, claim lifecycle, APIs UI/backoffice y trazabilidad/auditoria.
 8. Mantener este documento actualizado despues de cada milestone para evitar drift entre materiales de inversionistas e ingenieria.
-

2 English

2.1 Purpose

This document summarizes what the BRIDS app already has, what is partially built, and what should be built next. It is written for two audiences:

- Investors and strategic partners who need to understand technical maturity, product readiness, and execution path.
- Internal product, engineering, and operations teams that need one shared roadmap for the next delivery phases.

This is not a legal, tax, investment, or regulatory opinion. It is a technical and product-readiness brief based on the repository state and existing project documentation.

2.2 Executive Summary

BRIDS is currently a Solana-first real estate investment marketplace implemented as a Next.js fullstack application. The active product surface includes a public website, marketplace, property detail pages, wallet and federated authentication, protected investor dashboards, admin operations, collection/content management, checkout, purchase minting, compliance review, referrals, staking/unstaking, web push notifications, SEO/AI-readable content infrastructure, and governance automation.

2.3 Product Thesis

BRIDS is building a regulated, wallet-aware real estate investment platform where tokenized property inventory can be discovered, purchased, administered, and later operated through a transparent on-chain/off-chain lifecycle.

The current codebase supports four core investor-facing narratives:

1. Discover tokenized real estate opportunities through a marketplace and property detail experience.
2. Authenticate through either wallet-first SIWS or a lower-friction federated path that can later link to a wallet.
3. Execute crypto purchases already configured for USDC receipt, and add credit/debit card purchases through Sphere ramp under a controlled order model.
4. Operate holdings through protected dashboards, compliance gates, referrals, staking/unstaking, rent/portfolio modules, and notifications.

The technical direction is to turn this into a production-ready investment platform by hardening identity, compliance, payments, on-chain authority, observability, and marketplace performance.

2.4 Current Architecture

Layer	Current implementation
Frontend	Next.js App Router, React 19, Tailwind, Motion 12, Mapbox/React Map GL, responsive E2E coverage.
Backend	Next.js route handlers, server-only domain services in <code>lib/</code> , PostgreSQL repositories, idempotent purchase/checkout paths.
Blockchain	Solana devnet policy, Metaplex Core, Core Candy Machine, Umi, Solana Kit compatibility helpers, owner-freeze/stake flows.
Auth	Hybrid WorkOS + SIWS model, wallet linking, account repository, admin role derivation, protected routes.
Data	SQL migrations for marketplace entries, checkout, compliance, referrals, notifications, staking, and upload/edit sessions.
Admin Ops	Admin shell with assets, collections, compliance, dashboards, notifications, treasury, sales, monitoring, minting.
Quality	<code>npm run validate</code> , Vitest, Playwright, Synpress, docs governance, route/content/schema/SEO/operability checks.

2.5 What Is Done

2.5.1 Public Site and Marketplace

Status: implemented, with performance follow-ups and map bugs to fix.

Implemented capabilities:

- Public landing and institutional pages.
- Marketplace list, map, and detail routes.
- Marketplace data read model backed by persisted DB entries with snapshot fallback.
- Marketplace map surface using Mapbox/React Map GL with list-only fallback when map configuration is unavailable.
- Property detail sections for media, investment summary, economics, fees/return, documents/blockchain, governance, property information, and Google Maps preview.
- SEO, structured data, feeds, AI-readable endpoints, `11ms.txt`, `ai.txt`, knowledge routes, and content serializers.

Current limitations:

- The initial entry animation with Motion 12 still needs to be implemented to strengthen the app's first impression while preserving performance, accessibility, and reduced-motion preferences.
- Marketplace map performance is a known boundary, especially mobile JavaScript/LCP/TBT.
- Known map bugs must be fixed before investor demo and private beta readiness.
- Some map-related work remains in the P2 queue: lazy Mapbox boundary, Web Vitals recheck, coordinate validation, and degraded-state polish.

Primary evidence:

- `app/marketplace/page.tsx`
- `app/marketplace/[id]/page.tsx`
- `components/marketplace/*`
- `lib/property-marketplace-server.ts`
- `docs/features/feature-app-create-a-marketplace-3d-visual-bri-164.md`

2.5.2 Authentication and Account Model

Status: implemented foundation, production storage hardening required.

Implemented capabilities:

- Wallet-first SIWS authentication.
- WorkOS federated login foundation.
- Explicit wallet linking flow.
- Server-side account repository.
- Request authority model that distinguishes low-authority account access from wallet-authenticated financial/admin authority.
- Admin route protection through wallet-derived role checks.
- Referral intent capture during auth.

Current limitations:

- Session storage is process-local/in-memory in current documentation and must move to a shared persistent backend before horizontal production scaling.
- Account consolidation/recovery policies remain future controlled flows, not silent automatic merges.

Primary evidence:

- `app/api/auth/*`
- `lib/auth.ts`
- `lib/auth-store.ts`
- `lib/auth-session.ts`
- `lib/accounts/repository.ts`
- `docs/auth-flow.md`
- `docs/session-model.md`
- `docs/features/feature-shared-hybrid-auth-workos-wallet-bri-154.md`

2.5.3 Checkout, Orders, and Payments

Status: partially implemented.

Implemented capabilities:

- Checkout cart and order APIs.
- Order and payment attempt data model.
- Crypto purchase flow already implemented and configured so BRIDS receives USDC.
- Onboarding reward reservation/consumption integration.

Current limitations:

- The end-to-end credit/debit card purchase function through Sphere ramp still needs to be implemented.
- After Sphere ramp is implemented, card purchase and crypto purchase must be integrated into one coherent purchase/checkout experience.
- The final Sphere ramp contract, webhook model, settlement mapping, and operational support flow for card purchases remain pending.

Primary evidence:

- `app/checkout/page.tsx`
- `app/api/checkout/*`
- `lib/checkout-service.ts`
- `lib/checkout-repository.ts`
- `db/migrations/018_checkout_dual_payment.sql`

2.5.4 Purchase Minting

Status: crypto purchase implemented and configured for USDC receipt, production hardening pending.

Implemented capabilities:

- Quote, challenge, prepare, and submit APIs.
- Anti-bot challenge issuance and consumption.
- Rate limiting/idempotency storage.
- Candy Machine guard validation.
- Third-party signer support.
- SOL/USDC payment currency model.
- System configured so BRIDS receives USDC.
- Submitted transaction confirmation.
- Post-submit asset verification and reconciliation support.
- Compliance restriction guard in purchase challenge/prepare/submit routes.

Current limitations:

- Jupiter remains a proposed extension for broader crypto input, but the base crypto purchase flow and USDC receipt already exist.
- Production launch requires final treasury/payment destination policy, devnet-to-production migration plan, and live payment/asset evidence.
- The repository governance currently expects blockchain/NFT acceptance on devnet only unless production policy is explicitly approved.

Primary evidence:

- `app/api/purchase/*`
- `lib/purchase-service.ts`
- `lib/purchase-anti-bot.ts`
- `lib/purchase-attempts-repository.ts`
- `lib/purchase-webhook-reconciliation.ts`
- `docs/purchase-tracing.md`
- `docs/devnet-proof.md`

2.5.5 Admin Asset and Collection Operations

Status: implemented foundation with ongoing workflow improvements.

Implemented capabilities:

- Admin asset creation form.
- Asset upload workflow with signed URLs, client upload, finalize, edit sessions, orphan reconciliation, CDN purge, and PDF brief support.
- Import jobs and import preview.
- Admin collections list, detail, editing surfaces, documents, gallery, property information, location maps, and health workspace.
- Marketplace entry creation from admin flow.
- Collection health read models and manual review support.

Current limitations:

- Some admin screens still need ongoing usability, operational consistency, and internal team readiness improvements.
- Full non-technical admin playbooks and production operating procedures should be written before external launch.

Primary evidence:

- `app/admin/assets/new/page.tsx`
- `app/admin/collections/*`

- app/api/admin/assets/*
- app/api/admin/collections/*
- components/admin/*
- lib/admin/*
- db/migrations/002_asset_upload_contracts.sql
- db/migrations/003_asset_import_jobs.sql
- db/migrations/020_asset_upload_edit_sessions.sql

2.5.6 Admin Dashboard and Operations

Status: admin surface implemented, critical operating modules still required.

Implemented capabilities:

- Admin shell with navigation to dashboard, assets, collections, compliance, notifications, treasury, sales, monitoring, and minting.
- Base panels for executive summary, sales, treasury, distributions, notifications, and monitoring.
- Admin routes protected by server-side role enforcement.

Current limitations:

- The treasury module using Squads still needs to be implemented so the team can see balances, movements, approvals, and operating evidence from admin.
- The distribution module still needs to be implemented to integrate Squads with the freeze/unfreeze module, connecting eligibility, treasury, distributions, and claim state.
- The admin summary module needs stronger actionable KPIs: investor funnel, KYC conversion, sales, funds received, treasury state, pending distributions, claims, compliance queue, collection health, and operational alerts.
- The notifications module must integrate with CRM for campaigns, lead follow-up, segmentation, contact status, and outreach traceability.
- The compliance module must integrate KYC, Stripe Identity, first-party user data persistence, and compliance-specific RBAC permissions.

Primary evidence:

- app/admin/dashboard/page.tsx
- app/admin/treasury/page.tsx
- app/admin/distributions/page.tsx
- app/admin/notifications/page.tsx
- app/admin/monitoring/page.tsx
- components/admin/executive-dashboard.tsx
- components/admin/treasury-console.tsx
- components/admin/distributions-console.tsx
- components/admin/admin-notification-campaign-console.tsx

2.5.7 Metaplex Core, Candy Machine, and NFT Operations

Status: implemented on devnet with authority lifecycle hardening backlog.

Implemented capabilities:

- Metaplex Core collection and asset minting.
- Core Candy Machine deploy/submit/status/metadata/mint prepare/snapshot finalize flows.
- Economic AppData plugin payload for revenue share/yield fields.
- Authority lifecycle backend for transfer_delegate and appdata_authority.
- Authority registry and audit events.
- Devnet evidence for collection, asset, AppData, and authority operations.

Current limitations:

- Authority lifecycle has backend endpoints, but operational admin UI is pending.
- Registry/audit read endpoints are proposed but not implemented.

- Squads evidence validation currently validates declared structure/evidence, not full on-chain proposal execution proof.
- Legacy collection backfill into authority registry is pending.

Primary evidence:

- app/api/admin/core-candy-machine/*
- app/api/admin/metaplex-core/*
- lib/core-candy-machine-admin.ts
- lib/core-authority-lifecycle.ts
- lib/metaplex-core-admin.ts
- docs/nft-spec.md
- docs/authority-model.md
- docs/state-machine.md
- docs/devnet-proof.md
- docs/rotation-spec.md

2.5.8 Compliance, KYC, AML, and Suspension

Status: implemented operational review model, provider/live policy hardening pending.

Implemented capabilities:

- Compliance status projection: pending KYC, pending AML, pending review, fully verified, restricted AML, suspended.
- Admin compliance case queue.
- KYC decision, AML decision, suspend/unsuspend, notes.
- Audit event model for admin compliance mutations.
- AML screening enrichment through Helius-oriented service.
- Financial guardrails block restricted/suspended users from purchase routes.

Current limitations:

- Live KYC/AML provider integration and compliance operating policy should be finalized before production.
- A module is still needed to retrieve data from Stripe Identity and persist it in BRIDS' own database under an approved data minimization, security, and retention policy.
- An admin view is still needed so the authorized team can review customer KYC data and review status.
- RBAC must be extended with a specialized compliance reviewer/compliance agent role, separate from the general admin role, to operate KYC/AML reviews with scoped permissions.
- Manual review SLAs, escalation ownership, and data retention policy should be documented for operations and investors.

Primary evidence:

- app/admin/compliance/page.tsx
- app/api/admin/compliance/*
- app/api/internal/compliance/aml/screen/route.ts
- lib/compliance/*
- db/migrations/012_profile_kyc_compliance.sql
- db/migrations/013_aml_screening_enrichment.sql
- db/migrations/014_compliance_notes.sql
- docs/architecture.md

2.5.9 Protected Investor Dashboard

Status: implemented surface, some modules likely require real financial data integration.

Implemented capabilities:

- Protected layout and dashboard.
- Portfolio, rentas, referrals, stake, profile, history, and account support modules.
- Profile capture, KYC module, onboarding reward reminders, and guided tour.

- Referral landing and dashboard services.
- Wallet-bound web push enrollment path.

Current limitations:

- The user dashboard summary should be fed by real data from portfolio, purchases, rentas/claims, compliance, staking, and notifications modules.
- My Portfolio must be fed by real data from the database and on-chain reconciliation, not snapshots or placeholders.
- Rentas / Claim must be implemented so users can claim from there, request transfers, see claim/distribution states, and track progress using persistent data from BRIDS' own database.
- History must show the user's real operations: purchases, staking/unstaking, claims, transfers, relevant compliance events, and reconciled wallet transactions, fed by the other modules and persistent database records.
- Investor statements, tax documents, distribution history, and downloadable reporting remain future product work unless already handled outside this repo.

Primary evidence:

- `app/protected/*`
- `components/dashboard/*`
- `lib/referrals/*`
- `lib/onboarding-reward-service.ts`
- `db/migrations/023_referral_wallet_first_schema.sql`
- `db/migrations/024_onboarding_profile_completion_rewards.sql`

2.5.10 Staking and Unstaking

Status: implemented devnet-oriented owner-freeze action flow.

Implemented capabilities:

- Protected stake page.
- DAS inventory discovery for owned assets.
- Stake/unstake prepare and submit flow.
- Owner freeze/thaw transaction construction.
- Attempt persistence and reconciliation.
- Responsive E2E coverage for protected stake surface.

Current limitations:

- The stake/unstake event reconciliation and distribution preparation service still needs to be implemented: canonical RPC validation, Helius as observer/indexer, internal projection of validated events, KYC/profile eligibility reads, and treasury/Squads or period financial snapshot reads.
- The period pipeline still needs to be implemented to calculate eligible time, version distribution policy, handle `sync_pending/reconcile_pending` states, generate the output file for claim or later execution, and enforce idempotency/retries/mandatory reconciliation.
- The distribution microservice and claim lifecycle still need to be implemented: `pending`, `claimed`, `failed`, `disputed` states, UI/backoffice APIs, claim reconciliation with transaction evidence, and period reprocessing/recalculation.
- Traceability/audit architecture still needs to be implemented: treasury snapshots, NFT states, distribution runs, distribution items, and audit logs queryable by UI and external audit.
- Production semantics must be clearly defined: what staking means financially, when rewards accrue, how they are claimed, and how staking interacts with compliance and ownership transfer.
- Final investor-facing copy, claim/distribution states, and legal/regulatory framing should be locked before launch.

Primary evidence:

- `app/protected/stake/page.tsx`
- `app/api/protected/profile/stake/*`
- `components/dashboard/stake-module.tsx`

- lib/stake-service.ts
- db/migrations/031_stake_profile_persistence.sql
- docs/STAKE_AUDIT.md

2.5.11 Notifications and PWA

Status: implemented foundation with rollout gates.

Implemented capabilities:

- Web push subscriptions.
- Transactional delivery jobs.
- Internal enqueue/process APIs.
- Admin notification campaign preview/send.
- Notification health checks.
- PWA capability UI and installability shell.
- Wallet-bound subscription contract.

Current limitations:

- Production push delivery requires final VAPID/env configuration, audience policy, rate limits, and operational monitoring.
- Admin campaigns should remain guarded by preview hash and blocked-reason checks.

Primary evidence:

- app/api/notifications/*
- app/api/internal/notifications/*
- app/api/admin/notifications/*
- components/admin/admin-notification-campaign-console.tsx
- lib/notifications/*
- db/migrations/027_web_push_subscriptions.sql
- db/migrations/028_web_push_delivery_jobs.sql
- db/migrations/029_admin_push_campaigns.sql

2.5.12 Observability, Governance, and Quality

Status: implemented strong internal controls.

Implemented capabilities:

- Health endpoint and admin monitoring routes.
- Operability logging store and analytics event route.
- Security headers tests.
- Docs governance validation.
- PR policy and required docs checks.
- RFC and Linear planning scripts.
- Playwright/Synpress coverage for browser and wallet-critical flows.
- Top-level `npm run validate` gate.

Current limitations:

- Investor-facing production readiness should include uptime targets, alerting ownership, incident response, and data backup/restore evidence.
- Current local validation should be complemented by deployment-specific smoke checks for QA/RC/prod.

Primary evidence:

- app/api/health/route.ts
- app/api/admin/monitoring/*
- lib/observability/*
- scripts/ci/*
- scripts/rfc-new.js

- scripts/linear-plan.js
- docs/governance/*
- tests/*
- e2e/*

2.6 Maturity Matrix

Domain	Maturity	Investor-facing interpretation	Main next step
Marketplace discovery	Built	Real product surface exists.	Motion entry animation, map bugs, mobile performance, and lazy Mapbox boundary.
Property detail	Built	Assets can be presented with investment, docs, and governance context.	Final data completeness and compliance copy.
Wallet auth	Built	Strong wallet authority model exists.	Persistent production session store.
Federated auth	Foundation built	Lower-friction onboarding path exists.	Complete production WorkOS operations and recovery flows.
Checkout	Partial	Order model exists; card purchases need Sphere ramp.	Implement full Sphere ramp and unify card + crypto.
Crypto purchase mint	Built	Crypto flow is implemented and configured to receive USDC.	Production hardening, treasury policy, final evidence, and optional Jupiter.
Admin asset ops	Built	Internal team can manage inventory and metadata.	Operating playbooks and production hardening.
Admin dashboard	Partial	Admin shell exists; key operating modules are still pending.	Squads treasury, freeze/unfreeze distributions, CRM notifications, and actionable KPIs.
NFT/admin minting	Built on devnet	Core asset lifecycle exists.	Authority UI and read endpoints.
Compliance	Foundation built	Compliance gates are encoded into transaction paths.	Stripe Identity persistence, admin KYC view, and compliance RBAC role.
Investor dashboard	Surface built	User account and holdings UX exists.	Real-data summary, portfolio, rentas/claim, and history.
Staking	Base built on devnet	Asset action path exists; distributions and audit are pending.	On-chain reconciliation, claim state, distributions, and traceability.
Notifications	Foundation built	Re-engagement infrastructure exists.	CRM integration for campaigns and lead follow-up.
Observability/QA	Strong internal foundation	Engineering discipline is visible.	Production SLOs, alerting, deployment smoke gates.

2.7 Key Technical Gaps

1. Persistent session storage

Move auth/session state away from process-local memory into a shared production store. This is required for multi-instance deployments, restarts, and predictable auth behavior.

2. Checkout and payment hardening

Implement end-to-end credit/debit card purchases through Sphere ramp, define the final webhook/settlement contract, and unify the card flow with the already implemented crypto purchase flow.

3. Production treasury and payment policy

Validate production USDC settlement, define payment destinations, treasury ownership, environment-specific config, and audit evidence before live value movement. Jupiter remains an extension for broader crypto input.

4. Compliance operations

Implement first-party persistence for data retrieved from Stripe Identity, an admin customer KYC data view, a specialized RBAC role for compliance reviewer/compliance agent, manual review workflow, data retention, support escalation, and restricted-user behavior.

5. Authority lifecycle operations

Build admin UI for authority rotate/revoke/emergency rotate, implement registry/audit read endpoints, backfill legacy collections, and strengthen Squads/on-chain evidence validation.

6. Admin dashboard, treasury, distributions, and CRM

Implement Squads treasury, distributions integrated with freeze/unfreeze, CRM-connected notifications for campaigns/lead follow-up, and an admin summary with actionable KPIs for daily operations.

7. Marketplace entry animation, performance, and map bugs

Implement the initial Motion 12 entry animation, fix known map bugs, and defer/lazy-load heavy Mapbox work on mobile while preserving list-first usability, accessibility, and investor-grade visual quality.

8. User dashboard real data and investor financial reporting

Connect summary, my portfolio, rentas/claim, history, downloadable statements, and transfer tracking to the final production ledger, wallet/transaction reconciliation, and persistent database sources.

9. Staking distributions and audit trail

Implement the pending staking/distribution stack: canonical stake/unstake event reconciliation, period pipeline, KYC eligibility, treasury/Squads reads, distribution runs, claim lifecycle, UI/backoffice APIs, and audit logs.

10. Production observability

Define SLOs, deployment smoke checks, alerting ownership, incident response, backup/restore drills, and release promotion evidence.

2.8 Proposed Roadmap

2.8.1 Phase 0: Investor Demo Readiness

Target: 2 to 4 weeks.

Goal: make the current app demoable with high confidence and a clear story about what is real today.

Deliverables:

- Stabilize demo environment from `develop` or QA deployment.
- Prepare seeded marketplace inventory with complete property detail data.
- Implement the initial Motion 12 entry animation for the app entry experience.

- Fix known map bugs that affect the demo.
- Verify wallet login, marketplace, checkout preview, purchase quote, admin collections, compliance queue, and protected dashboard happy paths.
- Produce a demo script and investor FAQ from this document.
- Run `npm run validate` and critical Playwright evidence for the demo branch.
- Capture screenshots/video for pitch deck.

Exit criteria:

- Demo can be run end-to-end without local engineering intervention.
- Known gaps are clearly labeled as roadmap.

2.8.2 Phase 1: Production Foundation

Target: 4 to 8 weeks.

Goal: close the gaps that block a controlled private beta.

Deliverables:

- Persistent session backend for SIWS/WorkOS/session composition.
- Implement the full Sphere ramp flow for credit/debit card purchases and confirm webhook/idempotency behavior in deployed environment.
- Integrate the existing crypto purchase flow and the card purchase flow into one coherent checkout/purchase experience.
- Keep Jupiter as a proposed extension for converting additional crypto assets into USDC settlement.
- Compliance provider/live configuration, Stripe Identity persistence module, admin KYC view, and compliance reviewer/compliance agent RBAC role.
- Operating admin dashboard: Squads treasury, actionable KPIs, CRM notifications, and distribution preparation integrated with freeze/unfreeze.
- User dashboard with summary, portfolio, rentas/claim, and history fed by real persistent data.
- Production observability baseline: health checks, deployment smoke checks, alerting, incident owner, backup/restore plan.
- Initial Motion 12 entry animation, marketplace mobile performance pass, and Mapbox lazy boundary.

Exit criteria:

- Private beta can support real users in a controlled environment with production-grade auth and payment configuration.

2.8.3 Phase 2: Regulated Transaction Readiness

Target: 8 to 12 weeks.

Goal: prepare the app for live financial/on-chain operations subject to legal and compliance approval.

Deliverables:

- Final treasury and payment destination policy.
- Coherent integration between the existing crypto purchase flow and card purchases through Sphere ramp.
- Optional Jupiter integration for broad crypto input on top of the existing USDC receipt path.
- Live or pre-live on-chain acceptance plan with environment-specific signatures and fetched account state.
- Admin authority lifecycle UI and read endpoints.
- Legacy authority registry backfill.
- Compliance financial gates across all value-moving routes.
- Investor ledger/distribution source of truth for portfolio, rentas, history, and statements.
- Canonical stake/unstake reconciliation, distribution preparation service, distribution microservice, claim lifecycle, and traceability/audit for staking.
- Squads treasury, distribution module integrated with freeze/unfreeze, transfer requests, claim tracking, and persistent data for admin/user dashboards.
- Security review for wallet linking, admin routes, payment webhooks, signer flows, and authority rotation.

Exit criteria:

- The platform has auditable controls for identity, compliance, treasury, admin authority, and transaction execution.

2.8.4 Phase 3: Growth and Scale

Target: 3 to 6 months after private beta foundation.

Goal: expand inventory, improve conversion, and support repeat investor operations.

Deliverables:

- Multi-property inventory workflows with stronger batch operations and review queues.
- CRM-integrated notifications for campaigns, lead follow-up, transaction state, compliance status, distributions, and new offerings.
- Referral and onboarding reward optimization.
- Admin analytics for funnel, sales, compliance, and collection health.
- Actionable KPIs for treasury, claims, CRM, KYC conversion, sales, and operational health.
- Public marketplace SEO/content expansion.
- Institutional reporting package and data room exports.

Exit criteria:

- BRIDS can present the product vision, repeatable operating metrics, and controlled growth loops.

2.9 Pitch Deck Translation

This document can be converted into a pitch deck with the following slide structure:

1. Vision
 - Tokenized real estate investment marketplace with wallet-aware ownership and operational transparency.
2. Product Today
 - Marketplace, property detail, protected dashboard, admin operations, checkout, compliance, staking, notifications.
3. Technical Moat
 - Hybrid auth, Solana/Metaplex Core integration, admin minting, compliance gates, governance automation, test coverage.
4. Platform Architecture
 - Next.js app, server domain layer, PostgreSQL, Solana devnet/on-chain services, WorkOS/SIWS, Sphere ramp, Jupiter, observability.
5. Current Readiness
 - Marketplace, admin operations, auth, checkout, compliance, NFT, staking, notification, and observability modules are already represented in the app.
6. Risk Controls
 - Compliance statuses, purchase restrictions, idempotency, admin RBAC, authority registry/audit, docs governance.
7. Roadmap
 - Demo readiness, production foundation, regulated transaction readiness, growth and scale.
8. Use of Funds
 - Production hardening, compliance integration, payment/on-chain readiness, security review, marketplace performance, operations tooling.
9. Near-Term Milestones
 - Private beta, controlled transaction readiness, admin authority operations, investor ledger/distribution reporting.
10. Ask
 - Fund the path from working platform to compliant, production-grade investment marketplace.

2.10 Recommended Investor Claims

Use these claims because they are supported by the repository:

- BRIDS has a working fullstack app with marketplace, admin, auth, checkout, compliance, NFT, staking, notification, and observability modules.
- The payment roadmap is focused on implementing full Sphere ramp card purchases and unifying them with the crypto purchase flow already configured for USDC.
- The platform has a Solana/Metaplex Core implementation with devnet evidence.
- The compliance roadmap includes first-party Stripe Identity persistence, an admin KYC view, and a specialized RBAC role for reviewers.
- The operating roadmap includes Squads treasury, distributions integrated with freeze/unfreeze, CRM for notifications/campaigns, and dashboards fed by real data.
- The engineering process includes automated validation, docs governance, unit tests, route tests, Playwright, and Synpress.
- The remaining work is focused on production hardening, compliance/payment readiness, operational controls, and scale.

Avoid these claims until the corresponding roadmap items are complete:

- Do not claim mainnet production readiness.
- Do not claim fully live regulated investment operations.
- Do not claim completed card/crypto checkout parity until Sphere ramp and the unified integration are implemented.
- Do not claim production-grade persistent sessions until the shared session backend is implemented.
- Do not claim complete compliance operations until Stripe Identity persistence, the admin KYC view, and specialized RBAC role are implemented.
- Do not claim the complete staking/distribution lifecycle until canonical reconciliation, period pipeline, distribution service, claim lifecycle, and traceability/audit are implemented.
- Do not claim complete operating dashboards until admin/user dashboards are connected to Squads treasury, distributions, claims, CRM, reconciliation, and real persistent data.
- Do not claim finalized treasury, distribution, tax, or investor statement infrastructure until the ledger/reporting source of truth is complete.

2.11 Evidence Index

Repository and governance:

- README.md
- package.json
- docs/governance/documentation-policy.md
- docs/governance/git-monorepo-policy.md
- docs/governance/security-quality-policy.md
- scripts/ci/check-required-docs.sh

Architecture and security:

- docs/architecture.md
- docs/auth-flow.md
- docs/session-model.md
- docs/authority-model.md
- docs/state-machine.md
- docs/threat-model.md
- docs/rbac.md

Blockchain/NFT:

- docs/nft-spec.md
- docs/devnet-proof.md
- docs/rotation-spec.md

- lib/core-candy-machine-admin.ts
- lib/core-authority-lifecycle.ts
- lib/purchase-service.ts
- lib/stake-service.ts

Marketplace and admin:

- app/marketplace/*
- components/marketplace/*
- lib/property-marketplace-server.ts
- app/admin/*
- components/admin/*
- lib/admin/*

Checkout, compliance, notifications:

- app/api/checkout/*
- lib/checkout-service.ts
- app/api/admin/compliance/*
- lib/compliance/*
- app/api/admin/notifications/*
- lib/notifications/*

Testing:

- tests/*
- e2e/*
- npm run validate

2.12 Immediate Next Actions

1. Review this document with founders/product and soften any business claims that need it.
2. Turn the pitch deck translation section into a deck outline with screenshots from the live app.
3. Create Phase 0 execution tickets for demo readiness, map bugs, and evidence capture.
4. Create Phase 1 engineering tickets for persistent sessions, full Sphere ramp implementation, card + crypto integration, Stripe Identity persistence, admin KYC view, compliance RBAC role, and marketplace performance.
5. Create admin dashboard tickets for Squads treasury, freeze/unfreeze distributions, actionable KPIs, CRM notifications, and KYC compliance module work.
6. Create user dashboard tickets for real summary, real portfolio, rentas/claim, transfer requests, and history reconciled with wallet/modules/database records.
7. Create staking/distribution tickets for canonical event reconciliation, period pipeline, KYC eligibility, treasury/Squads reads, distribution service, claim lifecycle, UI/backoffice APIs, and traceability/audit.
8. Keep this document updated after each roadmap milestone so investor materials and engineering reality do not drift.